COMP7005 Lab4 Report

Will Otterbein, A01372608

Building the Lab and Source

I have included a makefile for this lab, with an override for the default `all` target, so you should only have to run the make command.

```
make
```

The built binary will appear under a directory called `build`, it should be named `thread-pool`.

The source code for this lab is located underneath the `src` directory, `thread_pool.c` has the main source code & there is also the `tp_common.c` which has some helper functions (chiefly this contains initializers for my program state structures).

`main`

```
41      // initialize job structs
40      app.jobs = NULL;
39      app.jobs = (tp_job_t*)calloc_s(app.job_c, sizeof(tp_job_t));
38      if (app.jobs == NULL) {
37          goto error;
36      }
35      for (int i = 0; i < app.job_c; i++) {
34          tp_initjob((app.jobs + i), i);
33      }
32
31      // initialzie worker structs
30      app.workers = NULL;
29      app.workers = (tp_worker_t*)calloc_s(app.worker_c, sizeof(tp_worker_t));
28      if (app.workers == NULL) {
27          goto error;
26      }
25      for (int i = 0; i < app.worker_c; i++) {
24          tp_initworker((app.workers + i), i);
23          pcR = pthread_create(&(app.workers + i)->pid, NULL, tp_dowork,
22              (void*) (app.workers + i));
21          if (pcR != 0) {
20              fprintf(stderr, "failed to created thread for worker w/ id %d\n", i);
19              continue;
18          }
17          (app.workers + i)->created = true;
16      }
15
14      // join worker threads
13      for (int i = 0; i < app.worker_c; i++) {
12          if ((app.workers + i)->created != true)
11              continue;
10          pjR = pthread_join((app.workers + i)->pid, NULL);
 9          if (pjR != 0) {
 8              fprintf(stderr, "failed to join thread for worker w/ id %d\n", i);
 7          }
 6      }
 5
 4      // check that all jobs are done
 3      fprintf(stderr, "\nin main thread before exiting check that all jobs are done:\n");
 2      for (int i = 0; i < app.job_c; i++) {
 1          tp_printjob((app.jobs + i));
126     }
./src/thread_pool.c
```

The main function first processes the arguments strictly rejecting any invalid argument count (n != 3), and parsing the two arguments as integers.

Then main sets up the application state. To make things easy, jobs & workers are assigned types, `tp_job_t` & `tp_worker_t` respectively, and the application state is grouped under the `tp_state_t` struct instance `app`. Heap memory is allocated for all the jobs & workers, as per the inputs of the program, & written to `app.workers` & `app.jobs`. The inital worker & job counts are also written to the application state as `app.worker_c` & `app.job_c`. The completed job counter, `app.c_job_c` , is set to 0.

For each job, `tp_job_t` structs are initialized w/ ascending id number & the other fields are zeroed as per `tp_initjob`, written the the corresponding index of `app.jobs`.

For each worker, `tp_worker_t` structs are initialized w/ ascending id numbers & the other fields are initialized as per `tp_initworker`. Additionally, `pthread_create` is invoked for the thread, where the routine is specified as `tp_dowork` & to that routine the current job struct is passed. `pthread_create` writes the pid to the structs `pid` field. If `pthread_create` fails, we print this & the `created` field of the `tp_worker_t` instance remains `false`, or 0.

Now that the workers are running, main has to wait for all the workers to finish. The `app.workers` memory is iterated

over, & `pthread_join` is called for each worker (unless the worker `created` field is set to `false`). If there's an error joining the thread, we print this.

Finally we just have a simple sanity check to log all the jobs & confirm that their state is set to done. Also the heap memory allocated for `app.workers` & `app.jobs` is freed. In the regular case, the exit code is 0.

Like my other labs, error conditions jump to an `error` label, which also frees the `app.workers` & `app.jobs` memory (if the pointer is not the default uninitialized value of NULL) and the program returns a non zero exit code.

`tp_dowork`

```c
void* tp_dowork(void* arg)
{
    tp_worker_t* worker = (tp_worker_t*) arg;

    while (1) {
        pthread_mutex_lock(&tp_jobqueue_a); // so this blocks until the mutex is lockable

        if (app.c_job_c ⩾ app.job_c) {
            pthread_mutex_unlock(&tp_jobqueue_a);
            break;
        }

        // save the job address
        tp_job_t* j = ( app.jobs + app.c_job_c );
        // advance a processed counter, which is like advancing through
        // a queue
        // threads use this to get the currently unprocessed thing
        app.c_job_c += 1;
        pthread_mutex_unlock(&tp_jobqueue_a);   // release so other threads can do stuff

        // the simulated work
        // continues invoking until nansleep returns 0
        // other nanosleep returns write to `tp_worker_rem`
        int strack = 0;
        int strack_c = 0;

        struct timespec tp_worker_rem = { 0 };

        strack = nanosleep(&tp_worker_delay, &tp_worker_rem);
        do {
            if (strack_c == TP_MAX_NSLEEP_RETRY) break; // just in case
            strack = nanosleep(&tp_worker_rem, &tp_worker_rem); // keep sleeping until the entire
            if (strack ≠ 0 && errno ≠ EINTR)
                break; // this is a legitimate error case w/ `nanosleep`
            strack_c++;
        } while (strack ≠ 0);

        // after work is done log that job is processed
        j→done = true;
        fprintf(stdout, "worker=%d JOB_DONE job=%d\n",
            worker→id, j→id);

        // tp_printjob(j);
    }

    pthread_exit(NULL);
    return NULL;
}
```

This function is the thread routine.

For its argument it accepts (expects) a pointer to the `tp_worker_t` type for that worker (in a more complicated application this might be more useful, I just use it to get that workers ID for the log message).

The processing logic begins with the `while` loop. The immediate condition check against the job count, `app.job_c`, & processed job offset, `app.c_job_c`, stops the threads from processing after the entire queue of jobs has been processed.

The while loop ensures that a thread will keep accepting jobs as long as there are jobs to accept, meaning

that the same thread will be used to process multiple jobs.

Regarding the processing, to ensure mutual exclusion while the threads are dequeuing jobs, the shared mutex `tp_jobqueue_a` must be acquired. If this cannot be done, `pthread_mutex_lock` takes care of blocking the calling thread for us. Otherwise the lock is acquired, the address of the current job to process is saved by the acquiring thread using the processed job offset, who is also responsible to advance the processed job offset (which is effectively like dequeuing the job).

The method initializes a `tp_worker_rem` timespec struct instance, which is used to write `nanosleep` remainder time in order that sleep be reattempted until the entire duration of 2 seconds has elapsed.

Once the fake work (`nanosleep`) is complete, the thread exits.

Sleep retries a maximum of 10 times before the retry loop is broken out of (defined by `TP_MAX_NSLEEP_RETRY`).